WEEK-03

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

```
#include <stdio.h>

#define MAX_TASKS 10
#define MAX_TIME 50   // Limit simulation time

typedef struct {
    int id;
    int execution;
    int period;
    int remaining;
    int deadline;
    int next_release;
} Task;

int main() {
    int n, time_limit = MAX_TIME;
    Task t[MAX_TASKS];
    int schedule[MAX_TIME];

    printf("Enter number of tasks: ");
    scanf("%d", &n);


    for (int i = 0; i < n; i++) {
        t[i].id = i + 1;
        printf("Enter execution time and period for Task %d: ", i + 1);
        scanf("%d %d", &t[i].execution, &t[i].period);

        t[i].remaining = 0;
        t[i].next_release = 0;
        t[i].deadline = t[i].period; // first deadline
    }


    for (int time = 0; time < time_limit; time++) {


        for (int i = 0; i < n; i++) {
            if (time == t[i].next_release) {
                t[i].remaining = t[i].execution;
                t[i].deadline = time + t[i].period;
                t[i].next_release += t[i].period;
```

```
            }
        }


        int selected = -1;
        int earliest_deadline = 9999;

        for (int i = 0; i < n; i++) {
            if (t[i].remaining > 0 && t[i].deadline < earliest_deadline) {
                earliest_deadline = t[i].deadline;
                selected = i;
            }
        }

        if (selected != -1) {
            schedule[time] = t[selected].id;
            t[selected].remaining--;
        } else {
            schedule[time] = 0; // Idle
        }
    }


    printf("\nGantt Chart:\n|");
    for (int i = 0; i < time_limit; i++) {
        if (schedule[i] == 0)
            printf(" Idle |");
        else
            printf(" T%d |", schedule[i]);
    }


    printf("\n0");
    for (int i = 1; i <= time_limit; i++) {
        printf("   %d", i);
    }

    printf("\n");

    return 0;
}
```

Output:

```
PS C:\Users\SHALINI K A\Desktop\OS> gcc RateMonotonic.c -o RateMonotonic
PS C:\Users\SHALINI K A\Desktop\OS> ./RateMonotonic
Enter number of tasks: 3
Enter execution time and period for Task 1: 1
2
Enter execution time and period for Task 2: 3
4
Enter execution time and period for Task 3: 2
5

Gantt Chart:
| T1 | T2 | T1 | T2 | T3 | T1 | T1 | T2 | T1 | T3 | T1 | T2 | T1 | T3 | T3 | T1 | T1 | T2 | T1 | T2 | T1 | T2 | T1 | T2 | T3 | T1 | T1 | T2 | T1 | T3 | T1
 | T2 | T1 | T3 | T3 | T1 | T1 | T2 | T1 | T2 | T1 | T2 | T1 | T2 | T3 | T1 | T1 | T2 | T1 | T3 |
0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32
 33  34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50
```

LAB - PROGRAM - 3

3. Write a C program to simulate real-time CPU scheduling algorithm

a) Rate-monotonic

```
#include <stdio.h>
# define MAX_TASKS 10
# define MAX_TIME 1000

typedef struct {
    int id;
    int execution_time;
    int period;
    int remaining_time;
    int next_release;
}
Task;
//Function to calculate GCD
int gcd(int a, int b){
    while (b!=0){
        int t=b;
        b=a%b;
        a=t;
    }
    return a;
}
//Function to calculate LCM
int lcm(int a, int b){
    return (a*b) / gcd(a,b);
}
```

```
int main(){
    int n, hyperperiod;
    Task tasks[MAX_TASKS];
    int schedule[MAX_TIME];

    Printf("Enter number of tasks:");
    scanf("%d", &n);
    //Input
    for (int i=0; i<n; i++){
        tasks[i].id = i+1;
        Printf("Enter execution time & period
            time for task %d: ", i+1);
        scanf("%d %d", &tasks[i].execution_
            time, &tasks[i].period);

        tasks[i].remaining_time=0;
        tasks[i].next_release=0;
    }
    //calculate exact hyperperiod using LCM
    hyperperiod = tasks[0].period;
    for (int i=1; i<n; i++){
        hyperperiod = lcm(hyperperiod, tasks[i].
            period);
    }
    //Sort by period (RM priority)
    for (int i=0; i<n-1; i++){
        for (int j=i+1; j<n; j++){
            if (tasks[i].period > tasks[j].period
            {
                Task temp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = temp;
            }
        }
    }
```

```
for (int time=0; time< hyper period; time++)
{
    //release tasks
    for (int i=0; i<n; i++){
        if (time == tasks[i].next_release){
            tasks[i].remaining_time =
                tasks[i].execution_time;
            tasks[i].next_release += tasks
                [i].period;
        }
    }
    //select highest priority task
    int selected = -1;
    for (int i=0; i<n; i++){
        if (tasks[i].remaining_time>0){
            selected = i;
            break;
        }
    }
    if (selected != -1){
        schedule[time] = tasks[selected].id;
        tasks[selected].remaining_time--;
    }
    else {
        schedule[time]=0;
    }
}
printf("\n Gantt Chart:\n");
for (int i=0; i< hyper period; i++){
    if (schedule[i]==0)
        printf("Idle |");
    else
        printf("T%d |", schedule[i]);
}
```

```
printf("\n0");
for (int i=1; i<=hyperperiod; i++){
    printf("%d", i)
}
printf("\n");
return 0;
}
```

output

Enter number of tasks: 3
Enter execution time & period for Task 1:
1 2
Enter execution time & period for Task 2:
3 4
Enter execution time & period for Task 3:
2 5

Gantt Chart:

T1	T2	T1	T2	T1	T2	T1	T2	T1	T2	T1	T2
0	1	2	3	4	5	6	7	8	9	10	11

T1	T2	T1	T2	T1	T2	T1	T2
13	14	15	16	17	18	19	20

b) Earliest-deadline First

```
 #include <stdio.h>

#define MAX 10
#define TIME_LIMIT 50

typedef struct {
    int id;
    int execution;
    int period;
    int remaining;
    int deadline;
    int next_release;
} Task;

int main() {
    int n;
    Task t[MAX];
    int schedule[TIME_LIMIT];

    printf("Enter number of tasks: ");
    scanf("%d", &n);


    for (int i = 0; i < n; i++) {
        t[i].id = i + 1;
        printf("Enter execution time and period for Task %d: ", i + 1);
        scanf("%d %d", &t[i].execution, &t[i].period);

        t[i].remaining = 0;
        t[i].next_release = 0;
        t[i].deadline = t[i].period;
    }


    for (int time = 0; time < TIME_LIMIT; time++) {


        for (int i = 0; i < n; i++) {
            if (time == t[i].next_release) {
                t[i].remaining = t[i].execution;
                t[i].deadline = time + t[i].period;
                t[i].next_release += t[i].period;
            }
        }


        int selected = -1;
        int min_deadline = 100000;
```

```
        for (int i = 0; i < n; i++) {
            if (t[i].remaining > 0 && t[i].deadline < min_deadline) {
                min_deadline = t[i].deadline;
                selected = i;
            }
        }


        if (selected != -1) {
            schedule[time] = t[selected].id;
            t[selected].remaining--;
        } else {
            schedule[time] = 0; // Idle
        }
    }


    printf("\nGantt Chart:\n|");
    for (int i = 0; i < TIME_LIMIT; i++) {
        if (schedule[i] == 0)
            printf(" Idle |");
        else
            printf(" T%d |", schedule[i]);
    }


    printf("\n0");
    for (int i = 1; i <= TIME_LIMIT; i++) {
        printf("   %d", i);
    }

    printf("\n");

    return 0;
}
```

Output:

```
PS C:\Users\SHALINI K A\Desktop\OS> gcc EarliestDeadlineFirst.c -o EarliestDeadlineFirst
PS C:\Users\SHALINI K A\Desktop\OS> ./EarliestDeadlineFirst
Enter number of tasks: 2
Enter execution time and period for Task 1: 1
4
Enter execution time and period for Task 2: 2
5

Gantt Chart:
| T1 | T2 | T2 | Idle | T1 | T2 | T2 | Idle | T1 | Idle | T2 | T2 | T1 | Idle | Idle | T2 | T1 | T2 | Idle | Idle | T1 | T2 | T2 | Idle | T1 | T2 | T2 |
dle | T1 | Idle | T2 | T2 | T1 | Idle | Idle | T2 | T1 | T2 | Idle | Idle | T1 | T2 | T2 | Idle | T1 | T2 | T2 | Idle | T1 | Idle |
0   1   2   3   4   5   6   7   8   9   10   11   12   13   14   15   16   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32
 33   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48   49   50
```

b) Earliest Deadline First

```
# Include <stdio.h>
# define MAX 10
# define TIME_LIMIT 50

typedef struct {
    int id;
    int execution;
    int period;
    int remaining;
    int deadline;
    int next_release;
}
Task;
int main() {
    int n;
    Task t[MAX];
    int schedule[TIME_LIMIT];

    printf("Enter number of tasks:");
    scanf("%d", &n);

    for (int i=0; i<n; i++){
        t[i].id = i+1;
        printf("Enter execution time & period
            for task%d", i+1);
        scanf("%d %d", &t[i].execution, &t
            [i].period);
        t[i].remaining = 0;
        t[i].next_release = 0;
        t[i].deadline = t[i].period;
    }
```

```
    for (int time=0; time<TIME_LIMIT; time++)
    {
        for (int i=0; i<n; i++) {
            if(time == t[i].next_release){
                t[i].remaining = t[i].execution;
                t[i].deadline = time + t[i].period;
                t[i].next_release += t[i].
                    period;
            }
        }
        int selected = -1;
        int min_deadline = 100000;

        for (int i=0; i<n; i++){
            if(t[i].remaining > 0 && t[i].dead
                -line < min_deadline ){
                min_deadline = t[i].deadline;
                selected = i;
            }
        }
        if(selected != -1){
            schedule[time] = t[selected].id;
            t[selected].remaining --;
        }
        else {
            schedule[time] = 0;
        }
    }
    printf("\n Gantt chart : \n");
    for (int i=0; i<TIME_LIMIT; i++) {
        if(schedule[i] == 0)
            printf("Idle ");
        else
            printf("T%d ", schedule[i]);
    }
```

```
    printf("\n 0");
    for (int i=1; i<=TIME_LIMIT; i++)
    {
        printf("%d", i);
    }
    printf("\n");
    return 0;
}
```

output

Enter number of Tasks:2
Enter execution time & period for Task 1:
1 4
Enter execution time & period for Task 2:
2 5

Gantt chart

T1	T2	T2	Idle	T1	T2	T2	Idle	T1	Idle
0	1	2	3	4	5	6	7	8	9

T2	T2	T1
11	12	13

10/4/26

C) Proportional Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    int choice, ticket;
    int t = 0;
    srand(time(0));
    printf("Proportional Scheduling\n");
    printf("1. Enter Ticket Manually\n");
    printf("2. Generate Random Ticket\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        // Manual input
        printf("Enter ticket number (0 - 100): ");
        scanf("%d", &ticket);

        if (ticket >= 0 && ticket <= 74) {
            printf("Ticket %d → Executed by P1\n", ticket);
        }
        else if (ticket >= 75 && ticket <= 100) {
            printf("Ticket %d is → Executed by P2\n", ticket);
        }
        else {
            printf("Invalid ticket!\n");
        }
    }
    else if (choice == 2) {
```

```
        int n, i;
        printf("Enter number of time slots: ");
        scanf("%d", &n);
        for (i = 0; i < n; i++) {
            ticket = rand() % 101;  // 0–100

            if (ticket <= 74) {
                printf("Time %d: Ticket %d → P1 executes\n", t, ticket);
            }
            else {
                printf("Time %d: Ticket %d → P2 executes\n", t, ticket);
            }
            t++;
        }
    }
    else {
        printf("Invalid choice!\n");
    }
    return 0;
}
```

Output:

```
"C:\Users\SHALINI K A\Deskt
Proportional Scheduling
1. Enter Ticket Manually
2. Generate Random Ticket
Enter choice: 1
Enter ticket number (0 - 100): 12
Ticket 12 ΓåÆ Executed by P1
```

```
"C:\Users\SHALINI K A\Deskt
Proportional Scheduling
1. Enter Ticket Manually
2. Generate Random Ticket
Enter choice: 2
Enter number of time slots: 5
Time 0: Ticket 96 ΓåÆ P2 executes
Time 1: Ticket 22 ΓåÆ P1 executes
Time 2: Ticket 85 ΓåÆ P2 executes
Time 3: Ticket 31 ΓåÆ P1 executes
Time 4: Ticket 25 ΓåÆ P1 executes
```

c) Proportional scheduling

```
# Include <stdio.h>
# Include <stdlib.h>
# Include <time.h>

int main(){
    int choice, ticket;
    int t=0
    Srand (time (0));
    Printf ("Proportional scheduling\n");
    Printf ("1. Enter Ticket manually\n");
    Printf ("2. Generate Random Ticket\n");
    Printf ("Enter choice:");
    Scanf ("%d", & choice);

    if (choice ==1){
        Printf ("Enter ticket number (0-100):");
        Scanf ("%d", & ticket);

        if(ticket >=0 && ticket <=74){
            Printf("Ticket %d -> Executed
                by P1\n", ticket);
        }
        else if (ticket >=75 && ticket <=100)
            Printf ("Ticket %d -> Executed
                by P2\n", ticket);
        else {
            Printf ("Invalid ticket!\n");
        }
}
```

```
else if (choice ==2) {
    int n, i;
    Printf ("Enter number of time slots:");
    Scanf ("%d", & n);
    for (i=0; i<n; i++){
        ticket = rand () %101;

        if(ticket <=74){
            Printf ("Time %d: Ticket
                %d -> P1 executes\n", t,
                ticket);
        }
        else{
            Printf ("Time %d: Ticket
                %d -> P2 executes\n", t,
                ticket);
        }
        t++;
    }
}
else {
    Printf ("Invalid choice!\n");
}
return 0;
}
```

output

```
Proportional scheduling
1. Enter Ticket manually.
2. Generate random ticket
Enter choice: 1
Enter ticket number (0-100): 20
Ticket 20 executed by P1

Enter choice: 1
Enter ticket number (0-100): 85
Ticket 85 executed by P2

Enter choice: 2
Enter number of time slots: 5
Time 0: Ticket 37 P1 executes
Time 1: Ticket 58 P1 executes
Time 2: Ticket 13 P1 executes
Time 3: Ticket 83 P2 executes
Time 4: Ticket 87 P2 executes
```